API2Test — Quick Start Guide

Import · Generate · Execute · Deploy — inside VS Code

API2Test turns an API specification into **runnable integration tests**, generated in the language you choose — **C#, TypeScript or Python**. Import a spec, build up a Data Dictionary of the fields each endpoint uses, generate request classes and test cases (a single call or a multi-step chain), then **execute** them in a built-in sandbox or **deploy** them into your own project and CI. This guide takes you from install to your first executed test.

1. Install

Install the bundled extension package **Api2Test.vsix** (this pack ships extension version 0.2.62):

```
code --install-extension Api2Test.vsix
```

Or in VS Code: Extensions view (Ctrl+Shift+X) → the "…" menu → **Install from VSIX…** → pick the .vsix file. Installing starts a **60-day free trial** that unlocks everything — there are no feature tiers — and the status bar shows the countdown, for example **API2Test: 54d left**. After the trial a licence key is needed (Command Palette → **API2Test: Enter License Key**); keys are verified offline and one key covers VS Code and the Desktop app. Full install steps are in InstallationGuide.pdf.

2. First run — pick your language, then the sandbox

On first activation API2Test asks **which language to generate in**. The choice is locked for the workspace, so pick the language your team tests in (C# is the default if you dismiss the prompt). It then maps a **sandbox** folder — the default offered is %APPDATA%\Api2Test\sandbox — where **Execute** builds and runs your tests, deliberately apart from your own source.

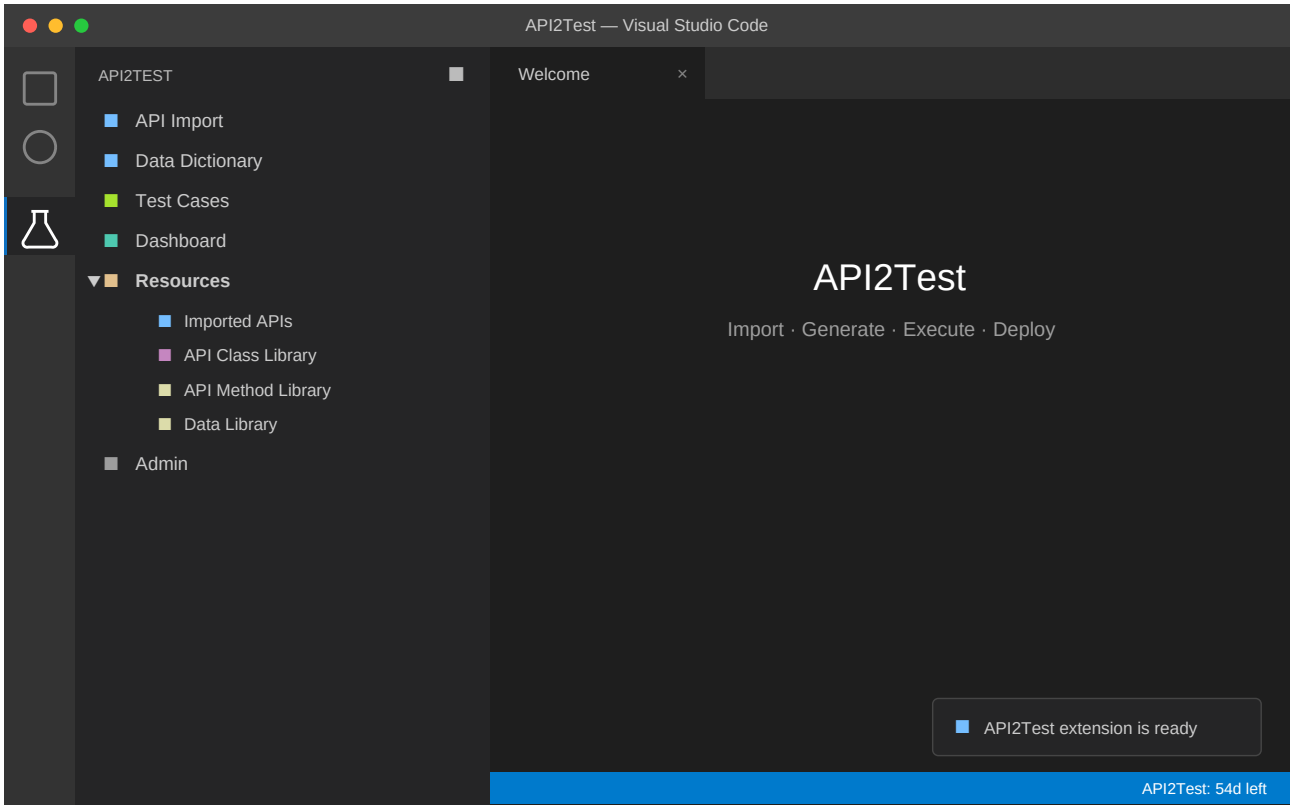
Generating code needs nothing installed — the generators are built in. **Executing it** needs the toolchain for the language you picked:

Language	Toolchain (machine-wide)	VS Code extension	Test framework + libraries
C#	.NET SDK (puts dotnet on PATH)	C# Dev Kit	MSTest + Bogus, already in the scaffolded ApiTests.csproj and r
TypeScript	Node.js LTS + npm	Vitest	npm install once in the sandbox typescript folder (vitest, typescrip
Python	Python 3 + pip (tick Add python.exe to PATH)	Python	pip install -r requirements.txt once in the sandbox python folder (

A **requirements page** then checks your machine row by row and reopens at every launch until everything is ✓. Each failing row says what to install and where. Only when all of them pass does the extension report that API2Test is ready.

3. Open the API2Test panel

Click the **flask icon** in the Activity Bar. The API2Test tree view opens with six entries, in the order you work through them:

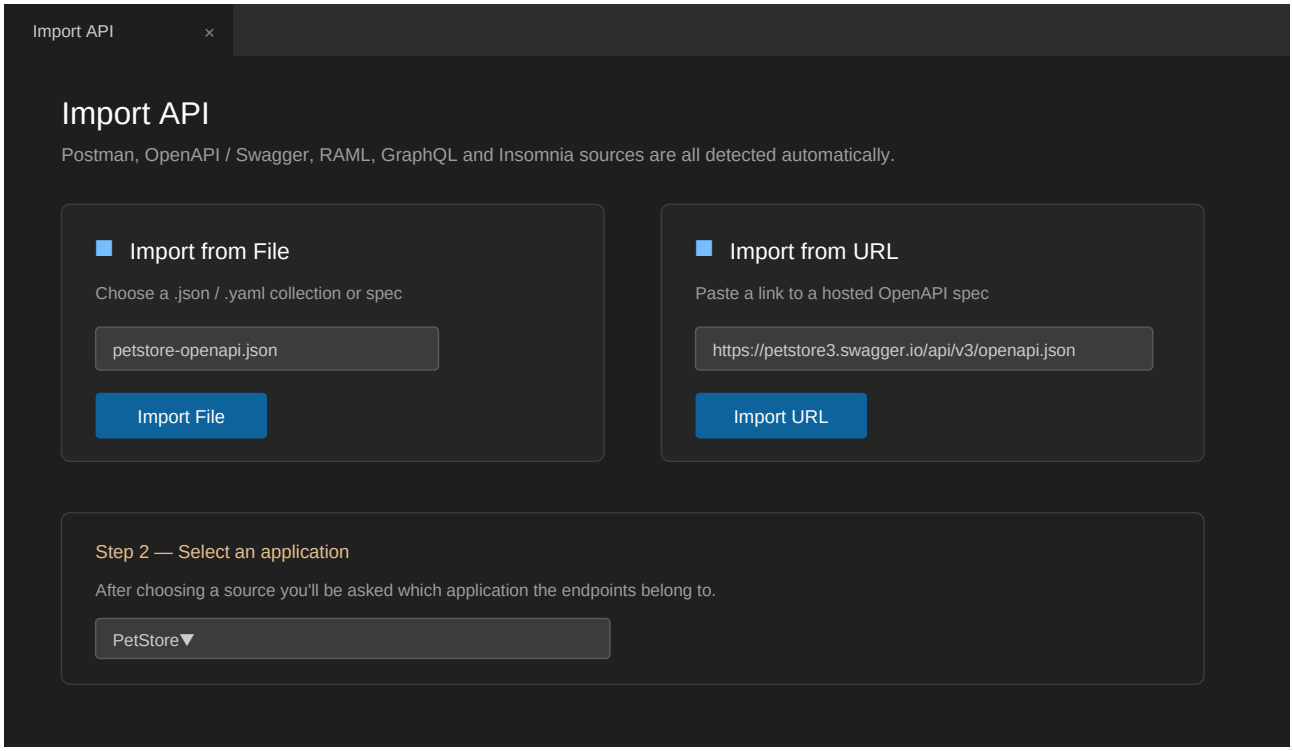


Entry	What it does
API Import	Import a spec — Postman, OpenAPI/Swagger, RAML, GraphQL or Insomnia
Data Dictionary	The fields each endpoint uses, and the data method that fills each one
Test Cases	Build, generate, execute and deploy test cases
Dashboard	The result of every run, with a drill-down report
Resources	Imported APIs and the Class / Method / Data libraries
Admin	Language, sandbox, deploy paths, requirements and licence

4. Import an API

Click **API Import** and choose a source. The format is detected automatically, then you pick the **application** the endpoints belong to — that name keeps each API separate all the way through to the generated folders and namespaces.

- **From File** — a Postman collection, an OpenAPI/Swagger .json / .yaml, or a RAML, GraphQL or Insomnia export.
- **From URL** — a link to a hosted spec.



Try it with the samples in the **Example Swagger Files** folder, or pick **From URL** and paste one of these live, free specs (no API key needed):

Public API	Spec URL (Import from URL)
Swagger PetStore (OpenAPI 3.0)	https://petstore3.swagger.io/api/v3/openapi.json
Swagger PetStore (Swagger 2.0)	https://petstore.swagger.io/v2/swagger.json
APIs.guru Directory (OpenAPI 3.0)	https://api.apis.guru/v2/specs/apis.guru/2.2.0/openapi.json

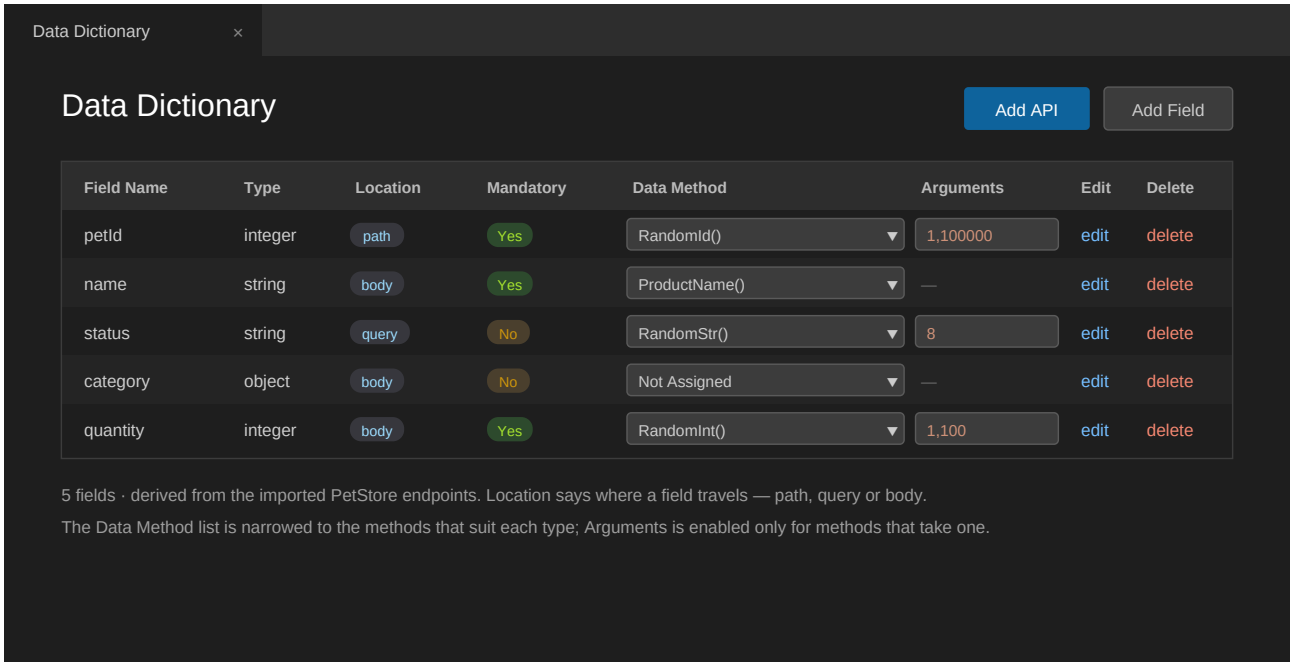
Then verify the result under **Resources** → **Imported APIs**.

5. Generate request classes

Under **Resources** → **API Class Library**, tick the endpoints you want and press **Generate Class** — one or a hundred in one go. Each class is named from the application, the verb and the path (POST /pet becomes PetStorePostPet), carries the spec field names as its JSON keys, and gets a ToJson() for use as a request body. The header keeps a running tally of generated, pending and failed. Change a field later and **Update & Generate** brings the class back into line.

6. Review the Data Dictionary

Open **Data Dictionary** to see every field the imported endpoints use — its type, its **location** (path, query or body), whether it is mandatory, and the **data method** that supplies its value. The method list is narrowed to the methods that suit the type, and arguments are enabled only for methods that take them, so DateStr becomes DateStr(-3) for three days ago. A field left as **Not Assigned** is generated without a default value.

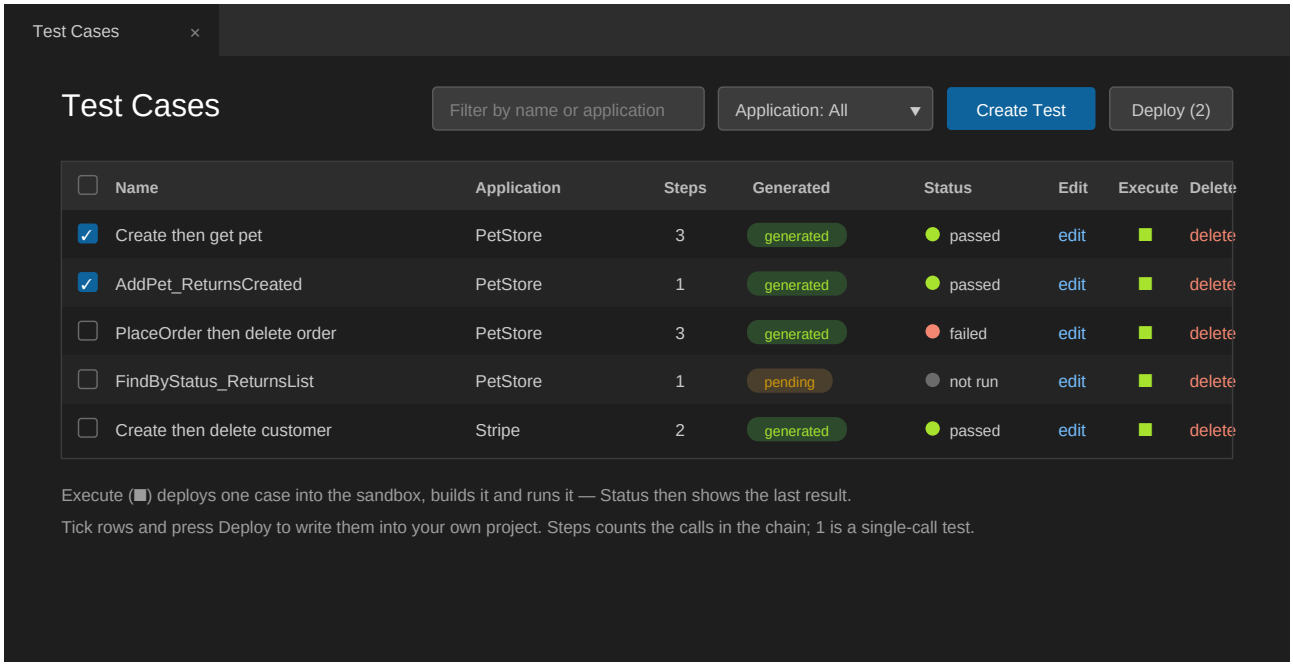


7. Build a test case — one call or a chain

Open **Test Cases** and press **Create Test**. Give the test a name, pick the application and the framework (MSTest for C#, Vitest for TypeScript, pytest for Python), then choose a **base path method** for the base URL and a **token method** for authentication. Now add steps with **Add class…** and **Add method…**, in the order they should run:

- **A method step** is one call from the API Method Library — PostJsonAsync, GetAsync, a Validate… check, an Extract… capture.
- **A class step** is the request body for that call, with any field pinned to a fixed value or to a value captured earlier.
- **IN** shows what a step needs (path parameters, body fields); **OUT** shows what it produces, where “store as” names a value for later steps.

A three-step PetStore chain reads: create a pet, validate the 200/201, capture the id it came back with, then fetch that pet by the captured id. Press **Generate & Save** and the test joins the list with its step count and status.



8. Execute it

Press **Execute** (the ■ on the row). API2Test copies the test, its classes and the libraries into the sandbox, builds, runs, and brings the result back — the row turns green or red, and **Dashboard** records the run with a report you can drill into, step by step. The same thing runs by hand in the sandbox folder with **dotnet test**, **npx vitest run** or **python -m pytest**.

9. Deploy into your own project

When a test earns its place in your repo, set the destinations once in **Admin** → **Deploy Paths**, then tick rows in Test Cases and press **Deploy**. API2Test writes a complete, compiling unit — the libraries, the request classes and the tests — into folders that mirror their namespaces:

```
Libraries/        -> Api2Test.Generated.Libraries
Classes/<App>/    -> Api2Test.Generated.Classes.<App>
Tests/<App>/      -> Api2Test.Generated.Tests.<App>
```

The **Example Generated Project** folder in this repository is exactly that output, for the PetStore API, and it builds with dotnet build.

Where your data lives

All API2Test data is stored as JSON at **~/.vscode/API2Test/data/** (Windows: C:\Users\<you>\.vscode\API2Test\data\). Files are created on first run and never overwritten, so your edits are safe across updates.

Troubleshooting

- **The requirements page keeps reopening** — a row is still red. It reopens at every launch until all of them pass, and each row says what to install.
- **Tree empty after import** — click Refresh at the top of the panel.
- **Import from URL fails** — make sure the URL returns the raw spec (JSON/YAML), not an HTML page.

- **Execute fails on the first run** — for TypeScript run npm install once in the sandbox typescript folder; for Python run pip install -r requirements.txt in the sandbox python folder.
- **A generated step carries a placeholder comment** — an argument was not supplied; reopen the test case and fill in the IN value.
- **Trial expired** — enter a key with API2Test: Enter License Key, or check Admin → Licence.

When you have finished trying API2Test, please complete the Feedback Questionnaire. Thank you!